ls
1
grep
1
true
2
nl
3
3
grep
4
nl
5
python
6
grep
6
nl
7
pytest
7
8
python
.../query.py
9
10
python
11
nl
12
python
12
13
echo
13
git
13
localization
patch
validation
general

```
.../alias datastructures.        for old_alias, new_alias in change_map.items():            if old_alias not in self.alias_map:                continue            alias_data = self.alias_map[old_alias].relabeled_clone(change_map)            self.alias_map[new_alias] = alias_data            self.alias_refcount[new_alias] = self.alias_refcount[old_alias]            del self.alias_refcount[old_alias]            del self.alias_map[old_alias]            table_aliases = self.table_map[alias_data.table_name]            for pos, alias in enumerate(table_aliases):                if alias == old_alias:                    table_aliases[pos] = new_alias                    break        self.external_aliases = {            # table is aliased or it's being changed and thus is aliased.            change_map.get(alias, alias): (aliased or alias in change_map)            for alias, aliased in self.external_aliases.items()        }"""s = s[:start] + new + s[end:]p.write_text(s)print("patched")
```

```
.../from django.db.models.sql.query import queryfrom django.db.models.sql.datastructures import basetablefrom django.db.models.sql.where import wherenode# create a query (model can be none for this test)q = query(model=none)# prepare alias_map with two aliases that would be involved in overlapping renameq.alias_map = {    't4': basetable('table1', 't4'),    't5': basetable('table1', 't5'),}q.alias_refcount = {'t4': 1, 't5': 1}q.table_map = {'table1': ['t4', 't5']}q.external_aliases = {'t4': false, 't5': false}q.where = wherenode()q.select = ()q.group_by = noneq.annotations = {}change_map = {'t4': 't5', 't5': 't6'}print("before:", list(q.alias_map.keys()), q.table_map)q.change_aliases(change_map)print("after:", list(q.alias_map.keys()), q.table_map)
```